10. Requirement Analysis

The successful implementation of the Medical Appointment & Prescription System is primarily determined by the quality of planning and execution. Accordingly, following the selected Waterfall Software Development Model, the requirement analysis is the stage, which builds the bridge between the general idea of the project and its comprehensive design.

Unlike the Software Requirements Specification Document, which describes the general scope of the software's requirements, the Requirement Analysis Document focuses on the description of each particular aspect of the application. In this way, the document ensures that each aspect of the software is thoroughly considered and designed, ranging from the actors' behaviors to the traceability of actions and the database's structure.

10.1 Stakeholder / Actor Analysis

The system contains four user categories. Each category has different access levels, objectives, and responsibilities. Thus, each category will be discussed separately.

i Administrator

Roles & Access: The administrator is the system's super user with master access. He/she manages the system's configurations, administers doctors directory, and oversees other users' activities.

Responsibilities:

- Ensuring the system's security features are working properly.
- Monitoring the system's performance to guarantee the system runs smoothly in terms of high-level database functions.
- Creating, updating, or deleting administrative and staff members' profiles while setting the right access levels.
- Setting up other configurations, general master data, and organizational level settings and ensuring their functionalities are running smoothly.
- Checking the system's data consistency at a high level and analyzing the system's administrative processes.

ii Receptionist/Admin Staff

Roles & Access: This category applies to front desk staff with limited access to the system in order to perform their duties.

Responsibilities:

- Doing away with the use of manual registers for patient appointments.
- Ensuring patient appointments are handled efficiently and patients' waiting time is minimized.
- Receiving patient appointments.
- Checking on the availability of appointments and ensuring that they are not scheduled on the same day and time.
- Updating the status of all appointments including those that have been cancelled, completed, or confirmed.
- Searching for patient profiles in the database.
- Verifying patient's background and ensuring that there are no mistakes.
- Maintaining up-to-date daily operation records.

iii Doctor / Medical Practitioner

Role & Access Details: Doctors can access patients' medical history, consultancies, and digital prescriptions.

Responsibility:

- The doctor conducts patient medical consultations and issues medical prescriptions digitally and accurately.
- Schedule Management Control: They manage their working hours by setting the start and end times of consultations and update the availability status daily.
- Clinical Background Review: They review the patient's medical background, treatment records, and clinical notes instantly when a medical procedure is initiated.
- Treatment Tracking & Documentation: They track the patient's treatment process, observe the recovery status, and update clinical notes after every session, consults, etc., and add additional notes after every patient's medical consultancy.

iv Patient

Roles & Access: This category applies to the system's end-users whose records are handled by other staff members in the hospital.

Responsibilities:

- Giving out his/her demographic information, contact address, and background to the administrators when registering.
- Giving out his/her symptoms to the selected physician during the consultation.
- Attending to the scheduled appointments on time and confirming the diagnosis.
- Taking medications as prescribed by the physician and adhering to the dosage instructions.
- Setting reminders for the medications.
- Getting the medication notes and printed reports (in pdf format) for reference.

10.2 Requirement Gathering Methodology

The process for capturing, analyzing, and documenting the requirements for the Medical Appointment & Prescription System comprised a wide range of investigative procedures and observations:

i Study of Existing System

The analysis of the conventional paper-based and manual hospital management system processes and procedures unearthed the following features that became a guiding star for our analysis:Manual recording of patient's records in paper registers and note-books and storing the same in hard copies cardboard files

- Manually marking consultations diaries by receptionists including maintaining wall calendars with ticked dates as a means of tracking occupied and available appointment slots
- Prescriptions are manually recorded in paper-pad by the medical personnel indicating the medicines and their dosages
- Easy physical misplacement of hard-copy patient records and consultation files and notes
- Easy overlaps of appointments because of no automated appointment tracking software solutions thereby creating inconvenience to patients
- Lack of automated reminder systems and alerts for scheduled follow-up visits, consultations, and medicine administration schedules.

ii Online Research Activities and Domain Analysis

The following online research activities were undertaken to gather pertinent information needed to design and develop the proposed computer-based Medical Appointment & Prescription System :

- Review of healthcare management concepts including computerizing existing manual small and medium clinic management information system that employ scattered paper registers for recording patient's personal and medical information
- Investigation and comparison of the technical design requirements of existing desktop applications using Core JAVA and Java-Swing, JDBC, and MySQL
- Determination of appropriate software security measures like using RBAC (Role-Based Access Control) and SQL PreparedStatement to secure local area network computer application against malicious electronic database infiltrations
- Human-Computer Interface (HCI) recommendations for creating navigational menus within desktop applications to reduce user training efforts
- Review of standard routines, functions and procedures in Java programming language for generating electronic clinical reports including the production of PDF reports using Java Technologies

iii Observation of Front Desk Activities in Clinics

It is extremely important to observe the front desk staff activities as well as patient –doctor interaction sessions so as to identify additional functionalities and requirements that were not captured in the interviews and system requirements analysis. Some of the observations made while observing the activities performed by the receptionist and patient check-in include:

- The traffic patterns at the front desk including the receptionist's interactions with patients over the phone as well as physical check-in and check-out activities
- The patient's physical check-in and check-out procedures from the clinic including walking directions as well as form filling and filing observation activities
- Doctor – Patient interaction and consultation procedures that require electronic medical history recall
- Doctor's prescription writing and handing over of the prescriptions to the patients

Observing these activities helped to unearth some implicit system requirements not captured in the system requirements specification document such as the receptionist's observation of consultation overlaps as well as the automatic appointment scheduling algorithms needed to prevent overlaps from occurring.

10.3 Detailed Functional Requirements (Module-by-Module and Actor Perspective)

- The functional requirements of the Medical Appointment & Prescription System are articulated below in relation to each of the proposed modules:

i.User Login and Roles

- **Administrator**
  - Be able to log in the master dashboard through administrator credentials and cross-verify with the database records
  - Be able to add, update or delete receptionist and doctor user accounts
  - Be able to add, update or delete role-playing credentials to ensure that access rights within the software are properly configured.
- **Receptionist**
  - Be able to log in the system software by using staff credentials and gain access to the front desk dashboard

- Be denied access to the admin dashboard where changes to user accounts and roles take place
- Be denied access to the doctor dashboard where prescription records are generated

- **Doctor**

  - Be able to access the software by using personal staff credentials to gain access to the doctor dashboard
  - Be denied access to the administrator dashboard where user accounts and roles are regulated

ii.Patient Registration, Records and Information

- **Receptionist**

  - Be able to navigate through the system interface and enter patient's personal information including their name, age, gender, contact address, contact phone number and blood group.
  - Have automatic warning mechanisms that prevent saving patient's personal information in case some of the fields are left blank.
  - Be able to pull up patient's personal information from the database using either the patient's file ID or name as the search criteria and update their contact address or phone number.

- **Doctor**

  - Be able to view patient's personal information and contact address from the database anytime the doctor needs to contact a patient by phone.

- **Patient**

  - Be able to have personal information registered in the database by the front desk receptionist during registration

iii. Doctor Registration, Records and Schedules

- **Administrator**

  - Be able to add or update personal information including doctor's specialization or contact address or phone number in the employee database

- **Doctor**

  - Be able to edit or update personal working schedules including weekly working hours and daily appointment time slots.
  - Be able to update the availability status to show either 'available', 'busy' or 'away' depending on the doctor's status.

- **Receptionist**

  - Be able to monitor doctor's availability calendar to ensure that appointments are only scheduled when a doctor is available

iv. Appointment Booking and Status Monitoring

- **Receptionist**

- Be able to navigate the appointment booking module by selecting patient ID, doctor ID, date and time of the appointment.
- Be notified by automatic system alerts in case of trying to book an appointment with a doctor who has already been scheduled at the proposed time.
- Be able to view and update appointment status including pending, confirmed, canceled or completed appointments.

- **Doctor**

  - Be able to view daily appointment queue and monitor statuses of confirmed, canceled, pending or completed appointment

v. Prescription, History and Reminder

- **Doctor**

  - Be able to generate electronic prescription records during or after a patient's consultation by indicating medicines, dosage, and intake instructions.
  - Be able to view prescription history including past medication records and associated clinical notes for every patient.

- **Patient**

  - Be able to view electronic medical slips that are generated by the doctor's prescriptions.
  - Be able to print out electronic PDF medical reports for patient's personal records.
  - Be able to receive medication reminders based on the prescribed dosage instructions.

10.4 Detailed Non-Functional Requirements

To ensure the Med Appointments & Prescriptions system performs reliably, safely and quickly in the day-to-day clinical working environment, the following quality standards have been set:

i High Performance

- **Database:** Any database query including searching for patient details, doctors' schedules through JDBC will load within less than 1.5 seconds.
- **GUI Interfaces:** Each Java Swing interface and transition between modules should be fully loaded within 1 second after clicking.
- **Concurrency:** Simultaneous appointment scheduling and prescription requests (by receptionist and doctor) should be processed by the backend without freezing more than 20 requests at a time.

ii Advanced Security

- **SQL Injection:** The database should by design (JDBC PreparedStatement) be completely protected from malicious SQL queries.
- **RBAC (Role-Based Access Control):** The login system should not allow receptionists or unauthorized staff to access doctor prescription modules under any circumstances.
- **User Passwords:** The password fields in the Users database should be encrypted with standard encryption algorithms.

iii Usability Standards

- **Training:** New users (receptionists) should require less than 30 minutes to learn and operate the new Java Swing interface.

- **Validation:** Each input field in forms should show validation errors or warnings in less than 0.5 seconds after the user clicks submit or ends focus.
- **Typography:** All forms and interface elements should use standard fonts (arial/sans-serif) with font size 12px to ensure readability.

iv. Reliability

- **Crash Avoidance:** The application should keep running without crashing during clinic hours with no less than 99.5% reliability.
- **Exception Handling:** Any runtime exceptions such as accidental closure of the database connection should result in a polite message to the user informing them to try again instead of crashing the desktop app.
- **ACID Compliance:** In case of an interrupted transaction in the MySQL database, the ACID properties should prevent the writing of corrupt data records.

v. Code Maintenance

- **Code Maintainability:** The java code should be written using object-oriented principles to separate UI modules (Java Swing) and database modules for future modifications.
- **Code Documentation:** Every java class and important methods should have documentation or comments in the code to allow easy maintenance by developers. At least 30% of code lines should be comments or documentation.

10.5 Use Case Diagrams & Descriptions

In order to illustrate the interaction between the described above system actors (Administrator, Receptionist, Doctor, and Patient) and identify the main areas of the Medical Appointment & Prescription System's functionality, the use cases are provided below with visual structural depiction and description.

i High-Level Use Case Architecture Outline

[Administrator] ----> (Manage System Users & Settings)
[Administrator] ----> (Manage Doctor Directory)

[Receptionist] ----> (Authenticate System Login)
[Receptionist] ----> (Register New Patient)
[Receptionist] ----> (Search & Update Patient Profiles)
[Receptionist] ----> (Schedule Appointment)
[Receptionist] ----> (Check Doctor Real-Time Availability)
[Receptionist] ----> (Update Appointment Status)

[Doctor] ----> (Authenticate System Login)
[Doctor] ----> (Configure Working Schedule)
[Doctor] ----> (Review Patient Medical History)
[Doctor] ----> (Create Digital Prescription)
[Doctor] ----> (Export PDF Medical Summary)

[Patient] ----> (Provide Registration Details)
[Patient] ----> (Attend Scheduled Consultation)
[Patient] ----> (Receive Digital Prescription & Reminders)

ii Detailed Use Case Descriptions

- **Use case 1: System authentication and role routing**

- **Actors:** Administrator, Receptionist, Doctor.
- **Precondition:** The user has opened the Java desktop application and has a valid account registered in the Users table of the database.
- **Postcondition:** The user is authenticated and redirected to their role-specific dashboard interface.
- **Main Flow**

  - User inserts their user name and password in the login form
  - System queries the JDBC prepared statement to authenticate the user
  - System determines the user's role (Admin, Receptionist, or Doctor)
  - System redirects the user to the dashboard interface

- **Alternative flow:** In case of invalid login credentials, system shows an error dialog "Invalid User Name or Password" and redirects the user to re-enter the credentials

- **Use case 2: Register new patient**

  - **Actor:** Receptionist.
  - **Precondition:** The Receptionist has logged into the front desk dashboard interface.
  - **Postcondition:** A new record is inserted into the Patients table of the MySQL database with a unique patient ID.
  - **Main Flow**
    - Receptionist opens the patient registration form
    - Receptionist fills in the personal information: Full Name, Age, Gender, Contact Number, Address, and Blood Group
    - Receptionist clicks the Save Patient button
    - System checks if all required fields have been filled
    - System inserts a record with a generated unique patient_id into the Patients table

  - **Alternative flow:** In case of empty fields, system shows an error message indicating which fields have not been filled

- **Use case 3: Schedule an appointment with clash detection**

  - **Actors:** Receptionist, Doctor.
  - **Precondition:** A patient is registered, and the doctor has defined working hours.
  - **Postcondition:** An appointment record is inserted into the Appointments table with the status "Pending" or "Confirmed".
  - **Main Flow**
    - Receptionist selects a patient_id and a doctor_id from the selection menus
    - Receptionist selects the appointment date and the available time slot
    - System runs the clash detection algorithm to determine whether the doctor is available for the selected time slot
    - If no clashes are detected, system inserts an appointment record and updates the status field

  - **Alternative flow:** In case of detected clashes, system declines the appointment and shows a warning dialog "Doctor is already booked for this time slot"

- **Use case 4: Issue a prescription and medical summary**

  - **Actor**: Doctor.
  - **Precondition**: The Doctor has logged into the dashboard interface while being on a consultation with the patient.

- **Postcondition**: A new prescription record is stored in the Prescriptions table for the current patient with a timestamp.
- **Main Flow**
  - Doctor views the patient's medical history from the current consultation window
  - Doctor fills in the patient's prescription form, defining the medicines, dosage, intake schedule, duration, and remarks
  - Doctor clicks the Save Prescription button to store the prescription information in the Prescriptions table
  - Doctor clicks the Export PDF button to view a digital medical summary sheet for the patient

- **Alternative flow**: In case of a connection loss with the database, an exception is thrown and caught by the try-catch block, notifying the doctor of the error and preventing the system from freezing the desktop application**.**

10.6 Requirement Prioritization (MoSCoW Method)

To control the scope during development, facilitate resource allocation, and set realistic time-to-delivery expectations for the Medical Appointment & Prescription System, the project will adopt the MoSCoW method of requirement prioritization (Must have, Should have, Could have, and Won't have):

i Must Have (Core Requirements – Basic MVP Set)

This category contains the key features that will be prioritized during development because without them, the software will cease to exist or serve its intended purpose. The following functionalities will be developed as a part of the core requirements:

- **User verification and routing:** JDBC PreparedStatement will be utilized to ensure secure user authentication and route traffic to the appropriate modules depending on the user's verified credentials (Administrator, Receptionist, Doctor).
- **Patient registration:** This feature encompasses the front-desk staff registration interface where patient information including name, age, gender, contact, address, and blood group will be captured and stored in a patient_id keyed table.
- **Doctor and schedule management:** This functionality allows the administrator to manage the doctor directory while the professional accesses and updates their working schedules.
- **Appointment booking with collision detection:** This core functionality enables the linking of patients to doctors' schedules while ensuring that no duplicate appointments are scheduled.
- **Prescription storage and management:** This will be the doctor's interface where medication notes and prescriptions will be stored and saved permanently in the MySQL database.
- **Database:** This is the main body of the application where the MySQL database will be designed around Users, Patients, Doctors, Appointments, Prescriptions, and other related entities.

ii Should Have (Nice-to-have but not Indispensable Features)

The following set of functionalities fall under the second priority level. They do not contribute directly to the software's core objectives but rather enhance the performance of the targeted users. These features can be eliminated during the development process without affecting the overall performance of the developed system:

- **Access to chronological list of prescriptions:** This functionality will enable doctors to access a chronological list of all medication notes and prescriptions made to any patient during their visit to the clinic.
- **Ability to track appointment status:** This feature will provide visual cues to track appointments through their status (pending, confirmed, completed, cancelled).

- **Input field audit:** This is a form of data validation technique that provides warning alerts when fields such as name, phone number, gender, and other demographic data are not correctly filled.
- **Export to PDF reports:** This feature will enable the user to capture and store medical reports and digital prescriptions in PDF format.

iii Could Have (Wish List Features)

This category of requirements is optional and should be considered for implementation only if there are remaining development hours. The features below are not detrimental to the system's overall functionalities but contribute to its effectiveness and ease:

- **Medicine timing reminder:** This functionality will be integrated into the system to remind patients of medication schedules depending on the prescribed dosage.
- **Smart patient search:** This feature will enable users to search for patients by name, phone number, or blood group.
- **End-of-day reports generation:** This is an extra feature that will be included in the system to allow staff to access end-of-day reports and patient visit statistics.

iv Won't Have (Excluded Features / Future Enhancements)

The following system features have been identified as suitable enhancements to the project but will be excluded from the current scope due to time constraints and the complexity of the application. Some of these functionalities include:

- **Online cloud hosting:** The current design of the application is a stand-alone desktop system. However, the system can be hosted on the cloud to allow remote access at a later stage.
- **Patients app:** This feature entails developing an android or iOS mobile application for patient use.
- **Online payments processing:** This will involve integrating existing online payment systems (e.g., Razorpay) to enable the processing of online payments for consultations.
- **Pharmacy stock and invoicing:** This functionality will require the design and development of features to track pharmacy stocks and generate invoices for medication dispensed.

10.7 Requirement Traceability Matrix (RTM)

The Requirement Traceability Matrix (RTM) below indicates the relationship between each core functional requirement of the Medical Appointment & Prescription System and the developed module as well as the relevant software validation test case to achieve comprehensive test coverage:

Req ID	Requirement Description	Target Development Module	Associated Validation Test Case
REQ-01	Secured credential authentication and routing according to role	User Authentication and Role Management Module	TC-Auth-01 (Valid Login) TC-Auth-02 (Invalid Password) TC-Auth-03 (Role Restriction)
REQ-02	Onboarding process, data validation, and record unique ID generation	Patient Registration and Records Management Module	TC-Pat-01 (Successful Registration) TC-Pat-02 (Missing Field

			Validation) TC-Pat-03 (Patient Record Lookup)
REQ-03	Doctor's profile maintenance and s chedule configuration	Doctor's profile maintenance and schedule configuration Doctor Management and Availability Module	TC-Doc-01 (Add Doctor Profile) TC-Doc-02 (Update Working Hours) TC-Doc-03 (Availability Toggling)
REQ-04	Appointment scheduling and automatic schedule clash detection	Appointment Scheduling and Status Tracking Module	TC-Sch-01 (Successful Booking) TC-Sch-02 (Double-Booking Clash Prevention) TC-Sch-03 (Status Lifecycle Update)
REQ-05	Digital prescription creation and chronolog ical storage of patient's prescription history	Prescription Management and History Module	TC-Rx-01 (Create Digital Prescription) TC-Rx-02 (Retrieve Patient History)
REQ-06	PDF report export and medicine reminder triggering	Reporting and Reminder Module	TC-Rep-01 (Export PDF Medical Summary) TC-Rem-01 (Trigger Reminder Schedule)

10.8 Data Requirements & Database Entities

In order to ensure data persistence and effective operation of the Medical Appointment & Prescription System, the data is stored on five core relational entities managed by a local MySQL database. The description below provides an insight into the data organization and the relationships between

the entities. The main attributes are listed together with the primary/foreign keys and their primary constraints.

Core Data Entities & Attributes

- **User Entity (Users)**
  - **Purpose**: This entity stores the credentials, user names, and permissions for each of the users of the system.
  - **Attributes:**
    - user_id (Integer, Primary Key, Auto-Increment)
    - username (Varchar, Unique, Not Null)
    - password_hash (Varchar, Not Null — securely hashed using bcrypt/SHA-256)
    - role (Enum / Varchar, Not Null — values: Admin, Receptionist, Doctor)
- **Patient Entity (Patients)**
  - **Purpose:** To store all personal details and contact information of the patients coming for treatment
  - **Attributes**
    - patient_id (Integer, Primary Key, Auto-Increment)
    - registered_by (Integer, Foreign key (references users.user_id), NOT NULL, ON DELETE RESTRICT)
    - full_name (Varchar, Not Null)
    - age (Integer, Not Null)
    - gender (Varchar, Not Null)
    - contact_number (Varchar, Unique, Not Null)
    - residential_address (Text, Not Null)
    - blood_group (Varchar, Nullable)
    - registration_date (Timestamp, Default Current Timestamp)
- **Doctor Entity (Doctors)**
  - **Purpose:** To store the details of the doctors working there like their qualifications, specialty, and availability
  - **Attributes:**
    - doctor_id (Integer, Primary Key, Auto-Increment)
    - user_id (Integer, Foreign Key referencing Users(user_id), Not Null)
    - full_name (Varchar, Not Null)
    - specialization (Varchar, Not Null)
    - contact_number (Varchar, Not Null)
    - consultation_fees(Decimal,Not Null)
    - working_hours (Varchar, Not Null — e.g., "09:00 AM - 05:00 PM")
    - availability_status (Boolean / Varchar, Default Active)
- **4. Appointment Entity (Appointments)**
  - **Purpose:** To store the booking information of a patient and a doctor with a particular date and time when the visit is planned, including the current status marker
  - **Attributes:**

- appointment_id (Integer, Primary Key, Auto-Increment)
- patient_id (Integer, Foreign Key referencing Patients(patient_id), Not Null)
- doctor_id (Integer, Foreign Key referencing Doctors(doctor_id), Not Null)
- appointment_date (Date, Not Null)
- appointment_time (Varchar, Not Null)
- status (Varchar, Not Null — values: Pending, Confirmed, Completed, Cancelled)
- created_at (Timestamp, Default Current Timestamp)

- **5. Prescription Entity (Prescriptions)**

  - **Purpose:** To keep electronic copies of the issued prescriptions, medication instructions,and ot her clinical documents related to the appointment
  - **Attributes:**

    - prescription_id (Integer, Primary Key, Auto-Increment)
    - appointment_id (Integer, Foreign Key referencing Appointments(appointment_id), Not Null)
    - patient_id (Integer, Foreign Key referencing Patients(patient_id), Not Null)
    - issue_date (Date)
    - doctor_id (Integer, Foreign Key referencing Doctors(doctor_id), Not Null)
    - medicines_and_dosage (Text, Not Null — detailed drug names, dosages, and frequencies)
    - clinical_notes (Text, Nullable)

- **6 Medical history entity (medical histories)**

  - **Purpose:** Capture physician notes during patient conversations. It includes symptoms, diagnoses, and treatment instructions that must be administered to the patient.
  - **Attributes**

    - record_id (Integer, Primary Key, Auto-Increment)
    - patient_id (Integer, Foreign Key referencing Patients(patient_id), Not Null)
    - doctor_id (Integer, Foreign Key referencing Doctors(doctor_id), Not Null)
    - recorded_at (Timestamp, Default Current Timestamp)
    - symptons (Text, Not Null - Stores descriptions of the patient's presenting complaints and clinical symptoms)
    - diagnosis (Text, Not Null - records the formal medical diagnosis determined by the attending doctor)
    - treatment_notes (Text, Not Null - contains auxiliary care instructions, lifestyle advice, or follow-up notes)

- **7 Medicine Reminder Entity (medical reminders)**

  - **Purpose:** Manages automated dosage alerts and tracking data to help patients adhere to their prescribed medication schedules.
  - **Attributes :**

    - reminder_id (Integer, Primary Key, Auto-Increment)
    - patient_id (Integer, Foreign Key referencing Patients(patient_id), Not Null)
    - prescription_id (Integer, Foreign Key referencing Prescription(prescription_id), Not Null)
    - scheduled_at (DateTime)
    - snoozed_until (DateTime)
    - status (Varchar, Not Null- “pending”,”taken”,”snoozed”)
    - interval_hours(Integer)

- **8 Medical Dose History Entiy (medical dose histories)**

  - **Purpose:** Tracks patient medication adherence by recording and logging if scheduled medicine doses are given or skipped.
  - **Attributes:**

  - history_id (Integer, Primary Key, Auto-Increment)
  - patient_id (Integer, Foreign Key referencing Patients(patient_id), Not Null)
  - prescription_id (Integer, Foreign Key referencing Medicine reminder(reminder_id), Not Null)
  - scheduled_at (DateTime)
  - given_at (DateTime)
  - status (Varchar, Not Null- “given”, missed”)

10.9 Constraints, Assumptions, & Limitations

In order to set the expectations and constraints during the development process of the Medical Appointment & Prescription System, it is essential to list the technical, operational, and time-related limitations linked to the project, along with the main assumptions, as demonstrated below.

i. Technical Constraints

- **Local On-Premise Deployment:** The software is designed as a local application, which means that the end-users cannot benefit from cloud-based synchronization, multi-branch replication, and remote access to the system
- **Single-Database Instance:** The database engine is also locally-hosted using MySQL, meaning that in case of a local computer or server malfunction, the data will be unavailable until the IT infrastructure is restored
- **Concurrency Limitations:** The software is primarily designed for small-to-medium-sized single-clinic environments with a limited number of workstations

ii. Time-Related & Resource Constraints

- **Academic Deadlines:** The time-scope of the project is limited by the academic calendar of the semester, implying that all features should be developed, tested, documented, and delivered within the established deadline using the MoSCoW prioritization model
- **Zero-Budget Policy:** The software is designed using open-source development tools and free database management systems, including Eclipse/IntelliJ IDE, and MySQL, due to the absence of budget allocated for commercial alternatives

iii. Operational Constraints

- **Role-Based Access Control:** Users should request dedicated access to the system, which eliminates the option to use self-registration mechanisms
- **Manual Front-desk Registration:** The reception desk staff is completely responsible for entering the personal information of patients, including their contact details

iv. Project Assumptions

- **Hardware Requirements:** The computer workstation, which will be used by doctors or administrators to access the system, should be equipped with operating system (preferably Windows/Linux/macOS), Java Runtime Environment, and MySQL database engine
- **End-Users Proficiency Level:** Clinic staff is expected to have an intermediate level of computer literacy to be able to operate the desktop application efficiently

- **Power Supply Stability:** The organization is expected to have stable power supplies and network infrastructure available to support uninterrupted operation of the database, particularly during write operations

10.10 Feasibility Summary & Project Evaluation

The feasibility study of the Medical Appointment and Prescription System is based on four major criteria that ensure that the proposed system project is realized within the defined time frame and budget constraints.They include:

- **Technical Feasibility**:The project is very feasible technically because it will be developed using open source technologies such as the core Java, Java Swing for the Graphical User Interface (GUI), Java Database Connectivity for connecting application programs to databases in order to perform SQL queries via JDBC, and MySQL as the database management system that provide comprehensive documentation.
- **Economic Feasibility**: The project will have no economic implications because it will be developed using open source technologies and run on personal computers thus incurring no license fees.
- **Operational Feasibility**: The system will be very feasible operationally because Java Swing GUI and front desk procedures are easy to use and will be adopted by hospitals and medical professionals with ease.
- **Schedule Feasibility**: The project's schedule for implementation based on the Waterfall development approach is realistic because the project will be realized within a short time while leaving enough time for testing.